

DramaScope — Final Project Defense Summary

Project Overview

Project name: DramaScope: Intelligent Video Scene and Narrative Analysis

Project purpose

DramaScope is an automated pipeline for analyzing short video scenes. It combines two complementary analysis approaches:

1. **Google Cloud Video Intelligence**, which extracts measurable visual metadata.
2. **A Large Vision-Language Model**, which provides higher-level semantic interpretation.

Existing video-analysis tools commonly provide either low-level detections, such as objects and shot boundaries, or general natural-language descriptions.

DramaScope combines both into structured, validated, and reusable JSON outputs.

Defense-ready project explanation

DramaScope analyzes short video scenes by combining deterministic computer-vision metadata with semantic interpretation from a Large Vision-Language Model.

Google Cloud Video Intelligence detects shots, labels, objects, confidence values, and human-presence intervals. I transform this data into four features: shot frequency, object entropy, interaction density, and human presence ratio.

These features are normalized and used to calculate a Scene Complexity Score and classify the scene as Calm, Dynamic, Dense, or Static.

In parallel, representative video frames are sent to the GLM vision-language model. It returns a scene summary and structured information about the setting, visible entities, actions, emotions, and interaction level.

The final results are validated and stored in `VideoFeatures.json` and `VideoInterpretation.json`.

Main contribution

The main contribution is not training a new foundation model. It is the design and implementation of an explainable hybrid system that:

- Combines numeric and semantic scene analysis.
- Converts raw API detections into meaningful features.
- Produces structured LVLM outputs rather than only free text.
- Supports independent tuning of the GCP and LVLM layers.
- Records explanations for classification decisions.
- Handles malformed responses and temporary API failures.

The final report defines the project as a modular pipeline for measurable scene analysis, rule-based interpretation, and LVLM semantic understanding.

Dataset

The project uses the **ACCED**E — Affective Video Content Database.

- 570 local **.mp4** clips.
- Typical duration: approximately 8–12 seconds.
- Extracted from Creative Commons films.
- Includes people, actions, objects, environments, and different editing patterns.
- Smaller subsets were used for development and tuning.
- The final LVLM interaction evaluation used 40 clips.

Although ACCED

E includes affective annotations such as arousal and valence, these values were not used as direct inputs to the current pipeline.

Page 2 — Complete System Workflow

1. Video input

The user selects a local video.

Example:

```
python -m src.pipeline.run_full_pipeline ^  
--video "videos/videos/ACCED
```

E09230.mp4" ^
--mode full

The main execution modes are:

- **gcp** — GCP feature extraction and algorithmic interpretation.
- **lvlm** — reuse existing GCP features and run LVLM analysis.
- **full** — execute the complete pipeline.

The repository also supports batch execution over multiple videos.

2. Google Cloud analysis

The video is uploaded to Google Cloud Storage and analyzed by Google Cloud Video Intelligence.
The API extracts:

- Video duration.
- Shot boundaries.
- Labels.
- Object tracks.
- Confidence values.
- Person-related temporal information.

The processed result is saved in: outputs/<video_id>/VideoFeatures.json

Relevant scripts

- `src/gcp/upload.py` — uploads the video and returns its GCS URI.
- `src/gcp/analyze.py` — calls Google Video Intelligence.
- `src/gcp/feature_engineering.py` — transforms annotations into structured metadata.

3. Feature engineering

Four numeric features are calculated.

Shot frequency

$$\text{shot frequency} = \frac{\text{number of shots}}{\text{video duration}}$$

It represents editing pace.

- High value: frequent cuts.
- Low value: longer shots.

It does not directly measure physical movement inside a shot.

Object entropy

Shannon entropy is calculated over the distribution of detected object categories:

$$H(X) = -\sum_i p_i \log_2(p_i)$$

It represents object-category diversity.

A scene with many different object types generally has higher entropy than a scene dominated by one repeated category.

Interaction density

$$\text{interaction density} = \frac{\text{tracked object segments}}{\text{video duration}}$$

Despite its name, this does not represent social interaction. It represents visual busyness and tracked-object activity.

This must be distinguished from the LVLM's semantic `interaction_level`.

Human presence ratio

$$\text{human presence ratio} = \frac{\text{time containing detected people}}{\text{video duration}}$$

Overlapping person-presence intervals are merged before calculation.

It measures how human-centered the scene is, not how many people appear.

4. Normalization

The features use different numeric ranges, so they are normalized to $[0, 1]$.

Final ranges:

Feature	Range
Shot frequency	0–1
Object entropy	0–4
Interaction density	0–5
Human presence ratio	0–1

Normalization prevents features with naturally larger values from dominating the final score.

Page 3 — Complexity Score, Classification, and LVLM Scene Complexity Score

The normalized values are combined using:

$$\text{Complexity} = 0.35S + 0.35E + 0.20D + 0.10H$$

Where:

- S: shot frequency.
- E: object entropy.
- D: interaction density.
- H: human presence ratio.

The weights are:

Feature	Weight
Shot frequency	0.35
Object entropy	0.35
Interaction density	0.20
Human presence ratio	0.10

Shot frequency and entropy receive the highest weights because they represent editing pace and visual diversity. Human presence receives a smaller weight because a scene containing people is not necessarily complex. The weights are explainable heuristic choices.

Narrative phase classifier

The classifier assigns one of four phases:

Dense

$\text{object_entropy} > 0.70$

AND $\text{interaction_density} > 0.65$

Represents visually crowded scenes with high object diversity and tracked-object density.

Dynamic

$\text{shot_frequency} > 0.35$

AND $\text{interaction_density} \leq 0.75$

Represents fast-paced scenes that are not visually dense enough for the Dense class.

Static

$\text{shot_frequency} < 0.20$

AND $\text{interaction_density} < 0.30$

AND $\text{object_entropy} < 0.50$

Represents scenes with slow pacing, low activity, and limited diversity.

Calm

Calm is the fallback when none of the other conditions are satisfied.

The classifier saves the tested conditions and reasons for the final decision, which makes the result explainable.

LVLM semantic analysis

Numeric features cannot determine what people are doing, what the setting is, or whether visible social interaction occurs. Therefore, frames are also analyzed by an LVLM.

The project initially used: glm-4.6v-flash

The later and final experiments used: glm-5v-turbo

Frame sampling

- Approximately one frame per second.
- Maximum of eight frames.
- Frames are extracted once and reused for both LVLM requests.

LVLM requests

The pipeline makes two requests:

1. A natural-language scene summary.
2. A structured JSON interpretation.

The structured result includes:

```
{
  "setting": "living room",
  "main_entities": ["man", "woman"],
  "actions": ["reading", "smiling"],
  "emotion_words": ["relaxed", "happy"],
  "interaction_level": 1,
  "interaction_evidence": "The woman looks up and smiles.",
  "summary": "Two people read together in a living room."
}
```

Interaction scale

Level	Meaning
0	No visible interaction
1	Weak, indirect, or one-sided interaction
2	Clear mutual interaction
3	Strong or intense interaction

The prompt was refined because early versions often failed to recognize subtle interactions such as glances, passive reactions, and shared activities.

Page 4 — Scripts, Reliability, and Results

Main scripts: Algorithm layer

- `src/algorithm/features.py` — builds the raw feature vector.
- `src/algorithm/normalize.py` — normalizes feature values.
- `src/algorithm/complexity.py` — calculates the complexity score and breakdown.
- `src/algorithm/phase.py` — applies phase-classification rules.
- `src/algorithm/validate.py` — validates numeric and structured outputs.

LVLM layer

- `src/lvlm/client.py` — extracts frames and calls the GLM API.
- `src/lvlm/prompts.py` — stores versioned prompts.
- `src/lvlm/parse.py` — removes code fences, extracts JSON, and parses responses.

Pipeline layer

- `src/pipeline/run_full_pipeline.py` — processes one video.
- `src/pipeline/run_batch_pipeline.py` — processes multiple videos.
- `src/pipeline/collect_lvlm_interaction.py` — updates interaction evaluation results.

Central configuration

`src/config.py` stores:

- Normalization ranges.
- Complexity weights.
- Phase thresholds.
- Frame settings.
- Model name.
- Prompt versions.
- Output filenames.

Output files

Each processed video receives:

`outputs/<video_id>/`

├── `VideoFeatures.json`
└── `VideoInterpretation.json`

`VideoFeatures.json` stores GCP-derived metadata.

`VideoInterpretation.json` stores:

- Raw and normalized features.
- Complexity score.
- Complexity breakdown.
- Narrative phase.
- Phase reasons.
- LVLM summary.
- Raw structured response.
- Parsed structured response.
- Error information.

Reliability mechanisms

The pipeline includes:

- Input-path validation.
- Numeric feature validation.
- LVLM response validation.
- Retry handling.
- Empty-response detection.
- JSON cleaning and parsing.
- Null-safe fallback fields.
- Detection of incomplete output files.
- Skip logic for already processed videos.
- `--force` for intentional reruns.
- Saving algorithm results even when the LVLM stage fails.

Thinking mode was disabled for `glm-5v-turbo` because it sometimes consumed the token budget and returned empty visible content.

Experimental results

Phase classification

Initial configuration:

Calm	Dynamic	Dense	Static
21	0	0	0

Final configuration:

Calm	Dynamic	Dense	Static
15	1	2	3

The first configuration demonstrated that the normalization ranges and thresholds did not match the dataset. Feature statistics were collected, ranges and thresholds were adjusted, and all four categories became reachable.

LVLM interaction evaluation

- Evaluation set: 40 clips.
- Exact matches: 25.
- Exact-match rate: 62.5%.
- Most remaining results differed by one interaction level and were considered partial matches.

The 62.5% value applies only to the LVLM interaction-level field against manually assigned labels. It is not an accuracy score for the entire pipeline.

Runtime

- Individual GCP analysis: approximately 18–33 seconds.
- Full pipeline for 40 clips: 28 minutes and 40 seconds.
- Average full processing time: approximately 43 seconds per clip.
- Offline unit tests: 38 passed.

Page 5 — Defense Questions and Limitations

Why combine Google Cloud and an LVLM?

Google Cloud provides consistent low-level metadata, while the LVLM provides scene context. One provides measurable structure; the other provides semantic interpretation.

Did you train an AI model?

No. The project uses pretrained models. The original work is in the architecture, feature engineering, normalization, scoring, classifier, prompt engineering, parsing, validation, orchestration, and evaluation.

Why use a rule-based classifier?

There was no sufficiently large labelled dataset for the custom Calm, Dynamic, Dense, and Static categories. A rule-based baseline is transparent, reproducible, and easier to explain than a model trained on a small subjective dataset.

How were the weights selected?

They were selected as interpretable heuristic weights. They were not statistically learned. Future work should optimize them using a larger independently annotated dataset.

Does interaction density measure people interacting?

No. It measures tracked-object segments per second and represents scene busyness. Semantic social interaction is represented separately by the LVLM's `interaction_level`.

Why sample only eight frames?

The clips are generally short. One frame per second provides reasonable coverage while controlling API cost, payload size, and runtime. The limitation is that brief actions between frames may be missed.

Why use two LVLM requests?

The free-text request produces a readable description, while the structured request produces fields that can be validated and processed programmatically.

A future optimization could use one structured request and generate the readable summary from its fields.

Is 62.5% a good result?

It shows moderate agreement on a subjective four-level task, but it is not a definitive benchmark. The labels were assigned manually, the evaluation set contained only 40 clips, and inter-rater agreement was not measured.

Why was no frontend created?

The project was designed as an R&D analysis tool. Development effort was prioritized toward the pipeline, experiments, validation, and structured outputs. A frontend could later consume the JSON files.

Main limitations

- Manual expected labels introduce subjectivity.
- Calm remains overrepresented because it is the fallback class.
- The numeric layer does not directly include optical flow, camera motion, audio, or facial-expression analysis.
- Fixed frame sampling can miss brief events.
- The project depends on external cloud APIs.
- Only a subset of the full dataset was manually evaluated.
- The reported interaction accuracy does not represent every project output.

Future improvements

- Add optical flow and camera-motion features.
- Select frames using shot boundaries or semantic changes.
- Create a larger evaluation set with multiple annotators.
- Measure inter-rater agreement.
- Learn or statistically optimize thresholds and weights.
- Compare GLM with open-source LVLMs.
- Reduce the two LVLM requests to one structured request.
- Add database storage and a web interface.
- Evaluate long videos by dividing them into scenes.

Final defense conclusion

DramaScope demonstrates that deterministic computer-vision metadata and LVLM semantic interpretation can complement each other. The GCP layer measures pacing, visual diversity, scene busyness, and human presence, while the LVLM describes setting, actions, emotions, and visible interaction.

The system's strengths are its modular architecture, structured output, explainable calculations, separate tuning modes, and reliability mechanisms. Its primary limitations are subjective evaluation labels, limited motion-aware features, and dependence on external APIs.

The project establishes a practical and reusable baseline for future video-understanding systems.

מדריך:

- להסביר בקצרה על הפרויקט, מה הוא עושה
 - להסביר מי קהל היעד, למי זה מיועד
 - למה זה אמור לשמש
 - השכבה הראשונה נותנת מידע נומרי וסטטיסטי
 - השכבה השנייה מתעסקת בהנגשה לקהל האנושי, טקסטואלית, גם לאנשים בלי ניסיון בתחום.
 - להסביר מי קהל היעד, מי מתכנן להשתמש בזה, מה המוצר
 - מה הבעיה הקיימת, מה הפתרון שאני מביאה לשולחן
- במאים לעיתים נהיים מאוד מקובעים על הסטייל והסגנון של הסצנות שהם רוצים/יש להם, שהם הרבה פעמים לא שמים לב שיש לא מעט בסצנות שלהם, שיש יותר מידי קאטים, או סיבוכיות, או שקט מידי וכולי. ואז זה נותן איזשהיא אינדיקציה לסצנות שהוא עושה ואז ככה (חוות דעת שלישית אובייקטיבית) הוא יודע מה לשפר או להחסיר מהסצנה.

מה הפרויקט עושה?

DramaScope הוא כלי מחקרי לניתוח אוטומטי של קטעי וידאו קצרים. המערכת מקבלת סרטון, מנתחת אותו בשתי שכבות: מצד אחד היא משתמשת ב־Google Cloud Video Intelligence כדי לזהות מידע מדיד כמו שוטים, אובייקטים, אנשים וקצב שינוי הסצנה, ומצד שני משתמשת ב־LVLMM כדי להבין ברמה סמנטית מה קורה בסצנה – מי מופיע, מה הפעולות, מה האווירה ומה רמת האינטראקציה בין הדמויות. בסוף המערכת מחזירה תוצאה מובנית בשני קבצי JSON עם נתונים כמותיים ותיאור משמעותי של הסצנה.

מה הבעיה הקיימת?

הבעיה היא שהכלים הקיימים בדרך כלל נותנים רק חלק מהתמונה. כלים כמו Google Video Intelligence טובים מאוד בזיהוי אובייקטים, שוטים ותנועה, אבל לא באמת מסבירים "מה קורה" בסצנה. לעומת זאת, מודלים מולטימודליים יכולים לתת תיאור עשיר יותר, אבל התוצאה שלהם פחות עקבית ופחות קלה למדידה ולהשוואה. הפרויקט שלי מחבר בין שני העולמות האלה למערכת אחת מובנית ומוסברת.

מה הפתרון שאני מביאה לשולחן?

הפתרון הוא pipeline היברידי: קודם נאסף מידע ויזואלי מדיד, ממנו אני מחשבת מאפיינים כמו **shot frequency**, **object entropy**, **interaction density** ו־**human presence ratio**. לאחר מכן אני מנרמלת אותם, מחשבת **Scene Complexity Score**, ומסווגת את הסצנה כ־Calm, Dynamic, Dense או Static. במקביל, ה־LVLMM מוסיף את השכבה הסמנטית – **interaction level**, **summary**, **setting**, **entities**, **actions**, **emotions**. כך מתקבל ניתוח שהוא גם כמותי וגם פרשני.

(**object entropy** - מדד לאי סדר של חפצים, אובייקטים)
(**interaction density** - דחיסות האינטראקציות, כלומר כמה האינטראקציה צפופה בסצנה פר שנייה ,
(**shot frequency** - תדירות הצילומים בסצנה)

מי קהל היעד?

הפרויקט נבנה ככלי מחקרי וניתוחי. קהל היעד המרכזי הוא חוקרים, סטודנטים, מפתחי AI/ML, אנשי data ו־content analysis, וגם יוצרים או אנשי מדיה שרוצים לקבל מידע מובנה על סצנות וידאו, ואף אנשים עם מוגבלויות בתחום הראייה. בדוח גם מצוין שהמערכת יכולה בעתיד לתמוך בתחומים כמו ניתוח מולטימדיה, כלי עזר ליוצרים ואפילו אפליקציות נגישות לאנשים עם לקויות ראייה.

מי מתכנן להשתמש בזה בפועל?

בגרסה הנוכחית, המשתמש הצפוי הוא יותר משתמש טכני או חוקר שמריץ את המערכת על סרטונים ומקבל קבצי `VideoFeatures.json` ו־`VideoInterpretation.json`. בעתיד אפשר לבנות מעל זה ממשק, API או מערכת גדולה יותר שתצרוך את התוצאות.

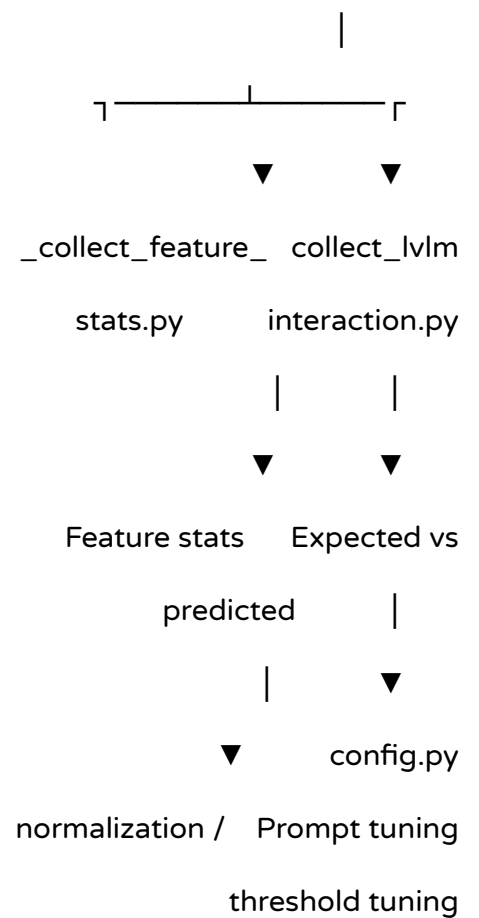
מה המוצר בפועל?

המוצר הוא מנוע ניתוח וידאו. הקלט הוא סרטון קצר (לא מוגבל לסצנה בעלת אופי דרמטי בניגוד לפרויקט קודם), והפלט הוא ניתוח מובנה שכולל גם סטטיסטיקות וגם משמעות סמנטית של הסצנה.

- הפרויקט שלי, DramaScope, הוא מערכת לניתוח אוטומטי של סצנות וידאו קצרות. הבעיה הנ"ל היא שכלים קיימים בדרך כלל נותנים מידע טכני כמו אובייקטים ושוטים, או תיאור כללי של הסצנה. אני משלבת Google Video Intelligence עם LVLM, כך שהמערכת מחשבת מאפיינים כמותיים כמו קצב שוטים ומורכבות סצנה, וגם מבינה ברמה סמנטית מי מופיע, מה קורה ומה רמת האינטראקציה. קהל היעד הוא בעיקר חוקרים, מפתחי AI, אנשי ניתוח וידאו ומולטימדיה, ובעתיד המערכת יכולה לשמש בסיס לאפליקציות נוספות בתחומי מדיה, נגישות ו־content analysis.



VideoInterpretation.json files



/pipeline – מנהל את כל התהליך.

/gcp – מוציא מידע ויזואלי ומדיד מהסרטון.

/algorithm – הופך את המידע לציונים וסיווגים.

/lvlm – מוסיף הבנה סמנטית של מה שקורה בסצנה.

/io_utils – תשתיות משותפות.

config.py – כל ההגדרות והפרמטרים במקום אחד.

Script	תיאור כללי	תפקיד ומטרה בפרויקט
<code>run_full_pipeline.py</code>	הסקריפט הראשי להרצת סרטון יחיד	מנהל את כל ה־GCP workflow, אלגוריתם, LVLM, ולבסוף מאחד את התוצאות ל־ <code>VideoInterpretation.json</code> .
<code>run_batch_pipeline.py</code>	הרצה על מספר סרטונים	מאפשר לבצע ניסויים והערכה על אוסף סרטונים בצורה אוטומטית, עם תמיכה ב־ <code>full</code> , <code>gcp</code> , <code>lvlm</code> , <code>skip</code> ו־ <code>force</code> .
<code>run_gcp_features.py</code>	הרצת שלב GCP בלבד	מחבר בין העלאת הסרטון, ניתוח ב־Google Video Intelligence ויצירת <code>VideoFeatures.json</code> . שימושי בעיקר לפיתוח ובדיקת שכבת GCP.
<code>collect_feature_stats.py</code>	איסוף סטטיסטיקות מהתוצאות	קורא קבצי <code>VideoInterpretation.json</code> , מחשב סטטיסטיקות של הפיצ'רים ועוזר לכוון <code>normalization thresholds</code> ו־ <code>ranges</code> .
<code>collect_lvlm_interaction.py</code>	הערכת תוצאות ה־LVLM	משווה בין <code>interaction_level</code> צפוי לחזוי ומעדכן את טבלת ההערכה. משמש ל־ <code>prompt tuning</code> ולבדיקת ביצוע המודל.
<code>verify_lvlm_fix.py</code>	בדיקת תקינות של תיקוני LVLM	משמש כ־ <code>smoke test</code> כדי לוודא שה־LVLM מחזיר פלט תואם לאחר שינויים או תיקונים. (בעת החלפת המודלים של VLM)
<code>run_video_analysis.py</code>	קובץ pipeline קטן/עזר	קובץ משני ולא חלק מרכזי מה־workflow הסופי; נותר כחל מהמבנה של הפרויקט.
<code>gcp/upload.py</code>	העלאת סרטון ל־Google Cloud Storage	הופך סרטון מקומי ל־GCS URI כדי ש־Google Video Intelligence יוכל לנתח אותו.
<code>gcp/analyze.py</code>	קריאה ל־Google Video Intelligence	שולח את הסרטון ל־API ומקבל את ה־ <code>annotations</code> : שוטיו, <code>labels</code> , <code>object tracks</code> ומידע זמני.

הופך את המידע הגולמי לנתונים מובנים: משך, שוטים, subject, entropy, human presence ratio, interaction density ושומר <code>VideoFeatures.json</code>.	עיבוד ה־annotations של GCP	gcp/feature_engineering.py
לוקח את <code>VideoFeatures.json</code> ומפיק את 4 הפיצורים המרכזיים: <code>shot_frequency</code>, <code>object_entropy</code>, <code>interaction_density</code>, <code>human_presence_ratio</code>.	בניית feature vector	algorithm/features.py
ממיר את הפיצורים לטווח אחיד <code>[0, 1]</code>, כדי שניתן יהיה להשוות ולשלב אותם בחישובים.	נרמול הפיצורים	algorithm/normalize.py
משקלל את הפיצורים המנורמלים לפי המשקלים ב־<code>config.py</code> ומחזיר גם score וגם breakdown של התרומה של כל פיצור.	חישוב Scene Complexity Score	algorithm/complexity.py
מסווג את הסצנה כ־<code>Calm</code>, <code>Dynamic</code>, <code>Dense</code> או <code>Static</code> thresholds, וגם מחזיר הסבר למה התקבלה ההחלטה.	סיווג Narrative Phase	algorithm/phase.py
בודק שהנתונים בכל שלב תקינים: מבנה JSON, ערכים מספריים, טווחי normalization ופלט LVLM. מונע מעבר נ מידע פגום בין שלבים.	שכבת validation	algorithm/validate.py
מחלץ frames מהווידאו, מקודד אותם, בונה בקשה ל־LLM מבצע inference ומטפל ב־retries, timeouts ושגיאות API	תקשורת עם מודל GLM	lvlm/client.py
מרכז את ה־summary prompt וה־structured prompt בגרסאות שונות, כדי לאפשר tuning, השוואה ושחזור ניסוי	שמירת ה־prompts	lvlm/prompts.py
מסיר code fences וטקסט מיותר, מחלץ JSON תקין וממיר את פלט המודל למבנה שניתן לבדוק ולשמור.	ניקוי וניתוח תשובת ה־LVLM	lvlm/parse.py
טוען משתנים כמו API keys ו־credentials מ־<code>env</code>, כדי שיהיו hardcoded בקוד.	טעינת environment variables	io_utils/env_utils.py

מספק טעינה בטוחה ואחידה של קבצי JSON לשימוש בכל חלקי המערכת.

פונקציות עזר ל־JSON

[io_utils/json_utils.py](#)

קובץ קטן ומשני הקשור לעיבוד וידאו; בפועל frame extraction המרכזי מתבצע כיום בתוך [.src/lv1m/client.py](#)

קובץ עזר ל־FFmpeg

[video/ffmpeg_extract.py](#)

מרכז את כל הפרמטרים החשובים: normalization ranges, weights, phase thresholds, frame settings, model name, prompt versions ושמות קבצי output.

קובץ הקונפיגורציה המרכזי

[config.py](#)

הסבר, פירוש - Annotations